MiniKV：面向大语言模型推理的 KV Cache 内存管理 （DDL: 10 月 31 号）

1 项目简介 2
2 背景知识 2
2.1 大语言模型是如何生成文本的 2
2.2 什么是 KV Cache 2
2.3 为什么 KV Cache 会成为一个数据结构问题 2
2.4 一种解决思路：把内存划分成固定大小的 Block 3
3 项目目标 4
3.1 项目的核心数据结构 4
涉及到的主要数据结构： 4
3.1.1 Array 表示物理内存 4
3.1.2 Linked List 保存一个 Request 使用的 Block 5
3.1.3 Hash Table 根据 Request ID 快速找到 Request 5
3.2 核心功能一：KV Cache 分配 6
3.2.1 实现两种内存分配策略 6
Strategy 1 连续分配 Contiguous Allocation 6
Strategy 2 分页式分配 Paged / Non-Contiguous Allocation 7
3.3 核心功能二：动态扩展 7
3.4 核心功能三：释放 KV Cache 8
3.5 核心功能四：请求查找 lookup 8
3.6 核心功能五：请求和物理内存状态显示 8
3.7 需要完成的项目任务 9
3.7.1 算法实现（在 matrix 系统上提供具体代码实现） 9
3.7.2 实验报告（在 matrix 系统上进行上传电子版文档） 9
Part 1 数据结构设计 9
Part 2 算法设计 9
Part 3 实验分析与结果 9
【说明 1：实验评估】 9
（1）不同工作负载下两个 Strategy 的表现 9
（2） Block Size 对系统的影响 10
【说明 2：算法分析】 10
ii) Fragmentation 指标 10
【说明 3：Matrix 任务与提交说明】 10
4 最终评分说明 11

1 项目简介

近年来，以 ChatGPT 为代表的大语言模型（Large Language Models，LLMs）被广泛应用于文本生成、代码生成、问答和智能助手等任务。虽然大语言模型背后的模型结构非常复杂，但在实际运行过程中，有一个非常重要的问题与**数据结构和内存管理**密切相关：

当一个大语言模型同时处理大量用户请求，并且每个请求产生的文本长度不断变化时，如何高效地管理有限的内存空间?

本项目将围绕这个问题，设计并实现一个简化版的 **KV Cache 内存管理系统 MiniKV**。重点是：

利用数组、链表等数据结构，设计一个能够动态分配、扩展和释放 KV Cache 内存的系统，并通过实验研究不同内存分配策略的特点。

2 背景知识

2.1 大语言模型是如何生成文本的

在使用 ChatGPT 等大语言模型时，我们输入一句话，例如： 法国的首都是, 模型可能依次生成： 巴黎。
从计算机的角度来看，文本会被拆分成一个个较小的单位，称为 **Token。**
为了简化以理解，我们假设：一个单词就是一个 Token。
例如： I love data structures
可以表示为： [I] [love] [data] [structures]
大语言模型生成文本时，并不是一次性生成完整答案，而是：
输入 → 生成 Token 1 → 生成 Token 2 → 生成 Token 3 → ...
随着生成的 Token 越来越多，模型需要保存越来越多的信息。

2.2 什么是 KV Cache

在大语言模型处理一个 Token 时，模型内部会产生一些后续计算还需要使用的信息。其中有两类重要的信息：
Key（K）和 Value（V）
模型会把这些信息暂时保存起来，这部分被称为：
KV Cache
可以简单地把 KV Cache 理解为：
为了避免重复计算，模型把之前处理过的 Token 的一些中间结果保存在内存中。

当模型继续生成新的 Token 时，就可以重复利用之前保存的信息。
因此： 生成的 Token 越多 → 需要保存的 KV Cache 越多 → 需要占用更多内存

2.3 为什么 KV Cache 会成为一个数据结构问题

假设一个 LLM 服务器有有限的内存空间。

例如，我们把内存抽象成 16 个位置：

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
空闲	空闲	空闲	空闲	空闲	空闲	空闲	空闲	空闲	空闲	空闲	空闲	空闲	空闲	空闲	空闲

现在有三个用户请求：

Request	需要的位置数目（个）
A	4

B	3
C	5

一种简单的分配方式是：

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	A	A	A	B	B	B	C	C	C	C	C	空闲	空闲	空闲	空闲

现在 Request B 完成了：

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	A	A	A	空闲	空闲	空闲	C	C	C	C	C	空闲	空闲	空闲	空闲

此时虽然还有很多空闲空间，但这些空间可能是分散的。

假设新的 Request D 需要 **6 个连续的位置**。如果没有连续的 6 个空闲位置，就可能无法分配。

这就是一个典型的**内存碎片（Memory Fragmentation）**问题。

2.4 一种解决思路：把内存划分成固定大小的 Block

为了解决上述问题，我们可以把内存划分成许多固定大小的 Block。

例如：

0	1	2	3	4	5	6	7
B0	B1	B2	B3	B4	B5	B6	B7

假设： Block Size = 4 Tokens，那么每一个 Block 最多保存 4 个 Token 对应的 KV Cache。例如，一个 Request 需要 10 个 Token：10 Tokens → 需要 3 个 Block

Block	使用的 Token 数
B0	4
B1	4
B5	2

注意：**这三个 Block 不需要在物理内存中连续。**

因此，可能出现：

B0	B1	B2	B3	B4	B5	B6	B7
A	A	B	C	C	A	C	D

但是，对于 Request A 来说，Request A：B0 → B1 → B5，这三个 Block 在逻辑上仍然属于同一个 Request。

这种思想与现代 LLM 推理系统中的 **Paged KV Cache / PagedAttention** 有密切关系。

3 项目目标

你的任务是设计并实现一个：**MiniKV —— 简化版 LLM KV Cache 内存管理系统**，系统需要支持多个 Request，并能够动态管理它们所使用的内存 Block。**项目核心思想：如何利用合适的数据结构管理不断增长的 KV Cache，并减少有限内存中的碎片和浪费?**

系统需要支持请求分配、动态追加、释放、查询、物理内存状态信息显示和请求状态信息显示六项操作：

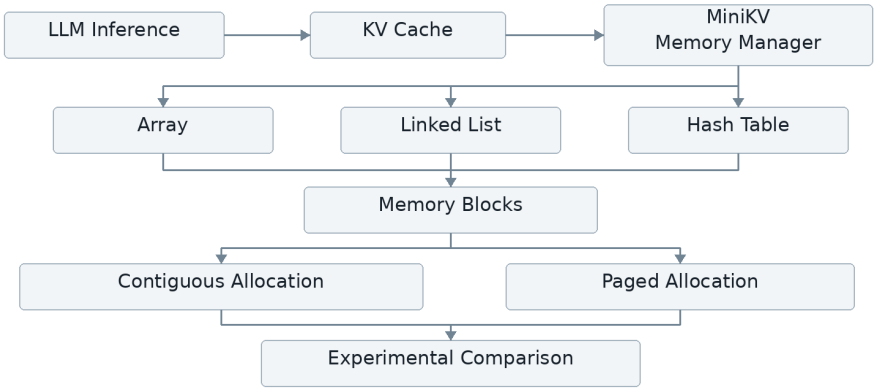
```
allocate(requestID, tokenCount)
append(requestID, newTokenCount)
free(requestID)
lookup(requestID)
displayMemory()
displayRequest(requestID)
```

其中，requestID 表示每个请求的 ID，tokenCount 表示当前请求所需要占用的 token 数，newTokenCount 表示当前请求额外申请的 token 数，上述的函数则具体分别表示：

- 为一个新的 Request 分配 tokenCount 大小的 KV Cache 空间；
- 当 Request 继续生成 Token 时为请求 requestID 增加 newTokenCount 大小的 KV Cache 空间；
- 当 Request 完成后释放对应的 requestID 的 KV Cache 空间。
- 查询请求为 requestID 的相关状态信息
- 显示物理内存的状态信息
- 显示请求为 requestID 的请求状态信息

3.1 项目的核心数据结构

本项目要求综合使用课程中学习的数据结构。核心任务可以概括为：



项目核心思想：如何利用合适的数据结构管理不断增长的 KV Cache，并减少有限内存中的碎片和浪费?

总的来说，这个课程项目里我们将会学习使用数组（Array）来表示物理内存，通过哈希表（Hash Table）来管理记录每一个请求，方便我们在大量的请求中进行快速查找，最后每个请求中会用一个链表（Linked List）来存储这个请求当前占用物理内存中的哪些 block。

这些主要用到的数据结构更加具体的用法参考如下：

3.1.1 Array 表示物理内存

整个物理内存使用数组表示。*例如： Block[] memory*

如果有 8 个 Block：

数组位置	Block
memory[0]	B0
memory[1]	B1

memory[2]	B2
...	...
memory[7]	B7

每个 Block 至少需要记录：

Block ID：Block 编号这里为了简化我们直接采用数组下标表示
owner：表示该 block 所属的 requestID，空字符串表示该 block 空闲
used：当前该请求 requestID 所使用的 token 数，如果 block 空闲则为 0

3.1.2 Linked List 保存一个 Request 使用的 Block

一个 Request 可能使用多个不连续的 Block，此时如果选用静态数组来表示会非常麻烦，一个是可能造成潜在的内存浪费，另一个是静态数组不够大的时候扩容需要花费较大的时间代价，因此我们常常使用链表这种结构来应对这种多插入删除少查询的情形。

例如：

对于一个请求来说，我们会希望它记录：

id：用于存储 requestID，作为请求的唯一标识

tokenCount：当前请求实际存储的 token 总数

head：指向分配给该请求的第一个 blockNode 节点

tail：指向分配给该请求的最后一个 blockNode 节点

hashNext：指向同一个哈希桶（可参考 3.1.3 的内容了解）内的下一个请求节点

这里我们就希望通过头节点 head 和尾节点 tail 来构建链表，进而存储对应请求所申请到的 block 信息，例如，对于 Request A 来说，这时候系统分配给了它 3 个 block，分别为 B0，B3，B6，那么就可以用链表表示为 B0 （head）→ B3 → B6 （tail） → NULL，其中 B0 就存放在 head 节点，B6 就存放在 tail 节点里。

3.1.3 Hash Table 根据 Request ID 快速找到 Request

系统中可能同时存在大量 Request，*例如： A123*、*B245*、*C781*、*D532*、*... 。*当我们需要对对应的请求进行操作的时候，比如后面提到的 KV Cache 的动态扩展问题，以及该请求结束后释放他所占用的 block，都要求我们需要先找到这个对应请求的状态信息，再访问他存储的 block 链表进行相关操作。但是当请求数量非常大的时候，如果我们仍然是一个一个的进行查找比较，这是非常耗费时间的。因此这时候一种经常采用的方法就是使用哈希表（Hash Table）来方便我们查找。

哈希表中最重的一个部分就是哈希函数，我们可以把它理解成一个位置计算器，负责告诉我们对应的 requestID 的请求应该放在哪个位置下。具体来说就是，输入请求的 id，它就会计算出一个数组下标，告诉我们应该去哪里查找这个请求。

例如，准备一个长度为 M 的数组，每个位置称为一个桶（Bucket）。我们可以采用一种简单的哈希函数：

$$h(id)=（requestID\ 中所有字符的\ ASCII\ 码之和）\bmod M$$

其中，ASCII 码是字符对应的整数，取模运算 % M 则保证结果落在 0 到 M−1 之间。例如，当 M = 5 时，请求 B245 对应的位置为：

$$h(B245)=(66+50+52+53) \bmod 5=1$$

因此，我们将 B245 的请求记录放入数组的第 1 号桶。以后需要查找它时，再计算一次哈希函数，就可以直接到这个桶中查找，无需从头检查所有请求。下面是 A123，B245，C781 在上面的哈希函数下对应的哈希桶：

Request ID	Hash Bucket
A123	0
B245	1

Request ID	Hash Bucket
C781	2

不过，不同的请求可能被分配到同一个桶，这种情况称为哈希冲突。例如，C781 的计算结果也是 1：

$$h(C781)=(67+55+56+49) \bmod 5=2$$

这里 C781 实际落在第 2 号桶；而 D532 的结果为 (68+53+51+50) % 5 = 2，因此它与 C781 发生冲突。解决方法之一是：将同一个桶中的请求用链表连接起来，这称为链地址法。就如在上面的 A123，B245，C781 后再增加 D532 的时候，此时哈希表的结构变化如下：

Hash Bucket	Linked List
0	A123->空
1	B245->空
2	C781->D532->空
3	空
4	空

在本项目中，结构体 Request 的 hashNext 表示 指向同一个桶中的下一个请求。查找时，我们先通过哈希函数找到桶，再沿着 hashNext 逐个比较完整的 id，直到找到目标请求或到达链表末尾。找到请求后，就可以读取它的 tokenCount，并通过 head、tail 访问它占用的 Block 链表。注意，hashNext 连接请求，Block 链表连接该请求占用的存储块索引信息。

当请求分布比较均匀时，每个桶中只需检查少量记录，因此查找通常比逐个遍历所有请求更快。上面的字符求和方法只是便于理解和实现，但并不保证分布均匀；实际系统通常会使用更合适的哈希函数，大家也可以探索选用不同的哈希函数带来的运行时间变化。

3.2 核心功能一：KV Cache 分配

实现： allocate(requestID, tokenCount)

推荐的操作流程：

- 通过哈希表查找 requestID，若已存在则失败；
- 计算所需 Block 数 ceil(tokenCount / S)，S 表示约定的 block 里含有的 token 数量，按指定策略（具体参考 3.2.1 和 3.2.2 中所描述的内容）检查能否获得全部空间；
- 成功后创建 Request 和 Block 链表，更新物理数组，并将 Request 插入对应哈希桶。tokenCount <= 0 或空间不足时返回 false，且不改变任何原有状态。成功返回 true。

例如：

Block Size = 4
allocate("A", 10)

因为： ceil(10 / 4) = 3，所以需要 3 个 Block。系统可能分配： Request A: B0 → B3 → B6，而不一定是： B0 → B1 → B2。两种情况都是合法的，具体取决于所使用的分配策略：

3.2.1 实现两种内存分配策略

为了研究不同数据结构和内存管理方法的效果，本项目要求在“KV Cache 分配“函数中实现两种策略。

Strategy 1 连续分配 Contiguous Allocation

其遵循一个请求必须获得连续的 Block。allocate 选择起点编号最小、长度足够的连续空闲区间。*例如： request A: B2 → B3 → B4 是合法的。但是：request A: B2 → B5 → B6 是不合法的。*

Strategy 2 分页式分配 Paged / Non-Contiguous Allocation

其遵循一个请求可以使用任意空闲 Block。但是为了统一，我们约定 allocate 每次优先占用 block 编号最小的区间。*例如： request A: B2 → B5 → B7 是合法的。*

逻辑上：

逻辑 Token 范围	物理 Block
Token 0 ~ 3	B2
Token 4 ~ 7	B5
Token 8 ~ 9	B7

物理位置可以是不连续的。

虽然分页式分配理论上可以使用任意空闲 Block，但是为了实现方便，我们作出如下约定：

统一采用最小 Block 编号优先：按编号从小到大依次选择空闲 Block，分配位置可以不连续，但不是随机选择。

例如：每个 Block 可以存放 4 个 Token，当前内存状态如下：

Block	B0	B1	B2	B3	B4	B5	B6	B7
状态	已占用	空闲	已占用	空闲	空闲	已占用	空闲	空闲

执行 allocate("A", 10) 时，需要 3 个 Block。按编号从小到大扫描：跳过已占用的 B0，选择 B1；跳过已占用的 B2，再选择 B3 和 B4。

Request A：B1 → B3 → B4

逻辑 Token 与物理 Block 的对应关系如下：

逻辑 Token 范围	物理 Block
Token 0 ~ 3	B1
Token 4 ~ 7	B3
Token 8 ~ 9	B4

虽然 B1 与 B3 不相邻，但仍可共同存放 Request A 的 Token。此时剩余空闲 Block 为 B6、B7，下一个请求需要新 Block 时，将优先分配 B6。

由于分页式分配策略的 block 编号不保证连续，这里为了统一，我们约定所有请求的 Block 链表按逻辑 Token 顺序排列：除尾 Block 外，其余 Block 均已存满。分页式追加时（对应 3.3 的内容），只把本次选出的新 Block 接到原链表末尾，不对原链表重新排序。

3.3 核心功能二：动态扩展

LLM 在开始生成文本时，通常并不知道最终会生成多少 Token。*例如： Request A：开始 6 Tokens → 继续 +4 Tokens → 继续 +5 Tokens。*因此，KV Cache 需要动态增长。

实现： append(requestID, newTokenCount)

例如： append("A", 5)

其中 newTokenCount 表示本次新增的 Token 数，而非追加后的总数。

推荐的操作流程：

- 先通过哈希表定位已有 Request；
- 再计算尾 Block 的剩余容量；若足够，只更新尾 Block 的 used 和 Request 的 tokenCount，不申请新 Block。否则，先确认所需新 Block 能全部获得，再填充尾 Block、接入新节点并更新 Token 总数。

更进一步，为了统一，我们对于 3.2 中的两种策略的追加规则约定如下：

策略	需要更新 Block 时的规则
连续分配	只能使用原区间右侧紧邻的 Block；遇到占用或越界即失败，不搬迁原请求
分页式分配	按物理编号从小到大选取当前空闲 Block，依次接到原链表末尾；空闲总数不足即失败

分页式追加操作的示例：原链表 B3 → B5，若 B1 已空闲，且本次需要一个新 Block，则追加后可为 B3 → B5 → B1。最小编号优先仅针对本次申请的新 Block，整条链表不必按物理编号递增。

注意 append 操作仅修改已有 Request，不新建 Request，也不重复插入哈希表。ID 不存在或 newTokenCount <= 0 时失败。同时，如果 append 操作成功返回 true，失败返回 false。

3.4 核心功能三：释放 KV Cache

当一个 Request 完成以后，它所占用的 KV Cache 就可以释放。

实现： free(requestID)

推荐的操作流程：

- 计算哈希值并沿冲突链找到 Request；
- 遍历其 Block 链表，将对应物理 Block 的 owner 清空、used 置 0，并释放 BlockNode；
- 从哈希冲突链中摘除 Request 并释放节点。删除桶首、中间或尾部节点时，均须保持同桶其他请求的连接。不存在时返回 false，成功返回 true；释放后的 ID 可以重新使用。

例如：

释放之前
request A： B0 → B3 → B6
request B: B1 → B5
执行： free(A)
之后： B: B1 → B5
同时： B0、B3、B6 重新变成空闲 Block，可以被其他 Request 使用。

3.5 核心功能四：请求查找 lookup

实现：lookup(requestID)
推荐的操作流程：
- 计算 ID 的哈希值定位桶
- 沿 hashNext 逐个比较完整 ID。找到时返回该 Request 的只读指针，未找到时返回 nullptr

哈希函数可自行选择，冲突采用链地址法处理。前面的 ASCII 求和取余仅用于演示，不要求使用该哈希函数。
这里的 lookup() 本身不输出。Matrix 主程序调用它：存在时输出 FOUND tokenCount blockCount，否则输出 NOT FOUND。tokenCount 是当前请求 requestID 占用的 Token 总数，blockCount 是当前请求 requestID 所占 Block 数，由主程序遍历 Block 链表得到。

3.6 核心功能五：请求和物理内存状态显示

1）输出请求的状态信息
实现：displayRequest(id)
推荐的操作流程：
- 通过哈希表定位 Request；

- 若不存在时输出 NOT FOUND，存在时输出 tokenCount blockCount blockID...。其中 Block 编号按逻辑链表顺序输出，相邻项用一个空格分隔。tokenCount 表示该请求占用的 token 数，blockCount 表示该请求占用的 block 数量。

2）输出物理内存的状态信息
实现：displayMemory()
推荐的操作流程：
- 按物理编号 0 至 N−1 遍历数组，依次输出各 Block 的所属 Request ID；空闲输出 -，相邻项用一个空格分隔。

注意，两个显示函数均不输出末尾换行，换行由 Matrix 主程序输出。

3.7 需要完成的项目任务

项目简化说明：本项目重点是 Data Structures，因此在本项目中，可以用简单的整数和元数据模拟 KV Cache。

例如：
Block 3
Request ID = A
Capacity = 4
Used Tokens = 3
不需要真正保存 Key 和 Value 向量。

3.7.1 算法实现（在 matrix 系统上提供具体代码实现）

程序至少需要提供以下功能：

allocate(requestID,tokenCount)
append(requestID,newTokenCount)
free(requestID)
lookup(requestID)
displayMemory()
displayRequest(requestID)

Matrix 系统中需要同学们完成实现指定策略类的构造、析构与六项操作；正确释放动态内存。实验报告中的测试序列、统计表格和图表可以自行设计。

3.7.2 实验报告（在 matrix 系统上进行上传电子版文档）

实验报告建议包含以下三部分。

Part 1 数据结构设计

描述项目实现的系统组织结构，要求提供该项目实现的数据结构系统设计图。

Part 2 算法设计

描述：allocate()、append()、free()等的实现方法，为主要算法提供伪代码。

Part 3 实验评估与算法分析

1、 实验评估，使用表格或图表展示： 不同 Block Size、不同 Allocation Strategy 下的实验结果。
（具体建议说明，可以参见下面**【说明 1：实验评估】**）
2、 算法分析：复杂度分析和 Fragmentation 指标
3、 （具体建议说明，可以参见下面**【说明 2：算法分析】**）

【说明 1：实验评估】
（1）不同工作负载下两个 Strategy 的表现

完成系统实现后，需要自行设计实验比较：**连续分配和分页式分配在不同工作负载下的表现。**
例如，可以设计如下操作序列：

allocate(A, 10)
allocate(B, 6)
allocate(C, 14)
free(B)
allocate(D, 8)
append(A, 5)
free(C)
allocate(E, 12)

对于相同的操作序列，分别运行： Contiguous Allocation（连续分配）和 Paged Allocation（分页式分配）

（2） Block Size 对系统的影响

还需要研究不同 Block Size 对系统的影响。自行设置实验，至少测试 Block Size 设置为 2、4、8、16 情况下的系统表现情况，给出实验结果及分析结论。

例如，一个 Request 需要 10 个 Token。

- 如果： Block Size = 4，则需要： 4 + 4 + 4，一共三个 block。其中，最后一个 Block 只有 2 个 Token 被使用，2 个位置空闲。
- 如果： Block Size = 16，则需要 1 个 block（16 个位置）： 10 个被使用，6 个位置空闲

因此：Block 越大并不意味着一定越好；Block 越小也不意味着一定越好。你需要通过实验分析不同 Block Size 带来的影响。

补充说明：关于实验分析方面，可以建议对于每一种策略和 Block Size，至少记录以下分析（也可自行设计）：
1. Allocation Failure：有多少次内存分配失败?
2. Memory Utilization：已经分配的内存中，有多少真正被 Token 使用?
3. Unused Space：由于 Block 没有完全使用而产生了多少空闲空间?

【说明 2：算法分析】

i) 时间复杂度分析：在实验报告中，需要分析主要操作的时间复杂度。
至少包括：

allocate()
append()
lookup(requestID)

对于每一个操作，需要说明：最好情况；最坏情况；平均/期望情况。

ii) Fragmentation 指标

Fragmentation 指标：该指标需要回答内存碎片情况如何？*例如，可以使用：Fragmentation =1 - (LargestFreeRegion / TotalFreeBlocks)*

iii) 除了 i)和 ii）之外，可以自行增加其他方面的分析，需说明你选择了什么指标，以及为什么选择这个指标。

【说明 3：Matrix 任务与提交说明】

本项目在 Matrix 系统（https://matrix.sysu.edu.cn）中分为以下三项任务。Project1-1 和 Project1-2 分别评测两种分配策略，Project1-3 用于提交整个 MiniKV 项目的实验报告。

- **Project1-1 MiniKV 之连续分配策略**；完成连续分配策略的代码实现，使每个 Request 占用连续的 Block，并支持空间申请、动态追加和释放等操作。按照该任务的题面、接口和输入输出要求提交代码，并通过评测。
- **Project1-2 MiniKV 之分页式分配策略**；完成分页式分配策略的代码实现，允许每个 Request 占用不连续的 Block。申请新 Block 时，按最小 Block 编号优先选择空闲 Block，并支持空间申请、动态追加和释放等操作。按照该任务的题面、接口和输入输出要求提交代码，并通过评测。
- **Project1-3 MiniKV 之实验报告**；完成前两项代码任务后，提交一份涵盖两种分配策略的实验报告。报告应包含数据结构设计、主要算法及伪代码、复杂度分析，以及不同工作负载和 Block Size 下的实验结果与比较分析；具体要求见本说明中的实验报告部分。

报告提交要求：报告须为 Word 文档，文件名必须为 **MiniKV.docx**，否则 matrix 系统上无法正常提交。请在报告首页注明自己的学号和姓名。

4 最终评分说明

部分	比例
Matrix 上 Allocate / Append / Free 等基础实现	20%
两种 Allocation Strategy 实现的正确性	40%
实验报告	40%

温馨提示：本次 project 看似比较庞大，但其实考察的内容比较基础，大家不要有太强的畏难情绪来看待。其中，增加了一个实验评估和算法分析部分，给大家留了比较大的空间自行设计，可以根据自己的思考来处理，只要逻辑自洽、思路清晰、结果和结论可靠可信、能够自圆其说即可。